Holdout Validation

(data split 기법)

미래연구소 16기 5주차

1. Hyperparameter - No Free Lunch

1) No Free Lunch

1> dataset 마다 최적의 hyperparameter가 다르다.
2> David Wolpert: The supervised learning no-free-lunch theorems.
3> tensorflow 공식문서: 적절한 크기나 구조를 결정하는 마법같은 공식은 없습니다.

2) 해결

1> iterative process가 필요하다.
2> 이를 효율적으로 하기 위한 방법: Holdout validation

https://en.wikipedia.org/wiki/No_free_lunch_theorem

1. Hyperparameter - No Free Lunch

TensorFlow 설치 학습 API 리소스 더보기 검색 Language GitHub

초보자

Keras를 사용한 ML 기본사항
- 기본 이미지 분류
- TF.Hub로 텍스트 분류
- 사전 처리된 텍스트로 텍스트 분류
- 회귀
- 과적합 및 과소적합
- 저장 및 로드

데이터 로드 및 사전 처리

에스티메이터

고급

맞춤설정

분산형 학습

과대적합을 막는 가장 간단한 방법은 모델의 규모를 축소하는 것입니다. 즉, 모델에 있는 학습 가능한 파라미터의 수를 줄입니다(모델 파라미터는 층(layer)의 개수와 층의 유닛(unit) 개수에 의해 결정됩니다). 딥러닝에서는 모델의 학습 가능한 파라미터의 수를 종종 모델의 "용량"이라고 말합니다. 직관적으로 생각해 보면 많은 파라미터를 가진 모델이 더 많은 "기억 용량"을 가집니다. 이런 모델은 훈련 샘플과 타깃 사이를 일반화 능력이 없는 딕셔너리와 같은 매핑으로 완벽하게 학습할 수 있습니다. 하지만 이전에 본 적 없는 데이터에서 예측을 할 땐 쓸모가 없을 것입니다.

항상 기억해야 할 점은 딥러닝 모델이 훈련 세트에는 학습이 잘 되는 경향이 있지만 진짜 해결할 문제는 학습이 아니라 일반화라는 것입니다.

반면에 네트워크의 기억 용량이 부족하다면 이런 매핑을 쉽게 학습할 수 없을 것입니다. 손실을 최소화하기 위해서는 예측 성능이 더 많은 압축된 표현을 학습해야 합니다. 또한 너무 작은 모델을 만들면 훈련 데이터를 학습하기 어렵울 것입니다. "너무 많은 용량"과 "충분하지 않은 용량" 사이의 균형을 잡아야 합니다.

안타깝지만 어떤 모델의 (층의 개수나 뉴런 개수에 해당하는) 적절한 크기나 구조를 결정하는 마법같은 공식은 없습니다. 여러 가지 다른 구조를 사용해 실험을 해봐야만 합니다.

알맞은 모델의 크기를 찾으려면 비교적 적은 수의 층과 파라미터로 시작해서 검증 손실이 감소할 때까지 새로운 층을 추가하거나 층의 크기를 늘리는 것이 좋습니다. 영화 리뷰 분류 네트워크를 사용해 이를 실험해 보죠.

`Dense` 층만 사용하는 간단한 기준 모델을 만들고 작은 규모의 버전와 큰 버전의 모델을 만들어 비교하겠습니다.

2. Holdout이란?

0) 어원

holdout : 협조를 거부 -> 숨겨둔 카드 => data를 다 쓰지 않고 검증(validation)을 위해 남겨둔다.

1) holdout validation

학습시키는 data 따로 , 검증하는 data 따로 split한다.

2) 사용하는 이유

학습한 data로 검증하는 건 unseen data에 대한 일반성을 확보하지 못하기 때문이다.

ex> 시험보는 학생에 비유하면 편하게 이해할 수 있다.
그동안 공부한 문제지로 실제 수능을 보면 객관적이지 못 함

3. Dataset 정의 1) training set

1> 용어 그대로 모델을 학습시키는 data

2> 최적의 parameter를 찾는 과정 = gradient descent = parameter update

3> fit 하는 과정

학생이 어떤 공부방법(hyperparmeter들의 조합)으로
문제지(training data)를 풀며
지식(parameter)을 학습하는 상황

3. Dataset 정의 2) validation set

1> 용어 그대로 내가 선택한 모델(hyperparameter의 조합)이 최적의 모델인지 학습하지 않은 data(validation set)으로 검증

2> 최적의 hyperparameter인지 검증 -> 결과에 따라 tuning함(수동)

3> 여러번 evaluate 할 수 있다.

4> 이 data로 gradient descent 하지 않는다.

학생이 그 동안 공부한 지식(parameter)을
모의고사(validation set)를 통해
선택한 공부방법(hyperparmeter들의 조합)이 맞았는지 검증하는 과정

3. Dataset 정의 3) test set

1> 용어 그대로 수차례의 검증을 마치고 최종 test에 사용하는 data

2> 결과에 따라 tuning 할 수 없음

3> 한 번만 evaluate 한다.

4> 이 data로 gradient descent 하지 않는다.

학생이 그 동안 지속적으로 검증한 공부방법(hyperparameter)를 통해
얻은 지식(parameter)을 가지고
수능(test set)을 통해 최종 평가를 받는 과정

3. Dataset 정의 4) distribution

training data = validation data = test data = unseen data

1> unseen data: 실제 device가 사용할 data

2> 최소한 validation = test = unseen은 성립해야한다.

3> 나중에 다시 자세히 다루게 될 예정

4. Holdout 2가지 방식

1) 2-way holdout = (training + test)

1> training data와 test data로만 분할해서 최종 test는 하지 않는다.
2> 사실상 test data가 3-way의 validation 역할을 한다.

2) 3-way holdout = (training + validation + test)

1> training data로 학습하고 validation으로 검증하는 수차례의 과정 가진 후
2> 최적의 모델을 찾았을 때 마지막으로 test set으로 한 번 더 검증

5. 실습 1) slicing 이용

Holdout Validation.ipynb에서 확인하겠습니다.

5. 실습 2) scikit learn의 train_test_split

Holdout Validation.ipynb에서 확인하겠습니다.